

Nama : Aditya Sechan Saputra

Nim : 2902795211

Pertanyaan 1

Konsep Dasar Linked List dan Perbedaannya dengan Array

1. Konsep Dasar Linked List

Linked List adalah struktur data linear yang terdiri dari beberapa elemen yang disebut **Node**. Setiap Node menyimpan:

1. Data
2. Referensi atau alamat menuju Node berikutnya

Contoh:

[Data | Next] → [Data | Next] → [Data | Next] → null

Pada **Singly Linked List**, setiap Node hanya memiliki satu pointer, yaitu next, yang menunjuk ke Node berikutnya.

Contoh class Node:

```
class Node {  
    int data;  
    Node next;  
}
```

Perbedaan Linked List dan Array

Aspek	Array	Linked List
Ukuran	Tetap	Dinamis
Penyimpanan	Kontigu/berurutan di memori	Tidak harus berurutan
Akses data	Cepat menggunakan index	Harus traversal dari awal
Insert	Lebih lambat karena pergeseran data	Lebih mudah dengan perubahan pointer
Delete	Perlu menggeser elemen	Cukup mengubah pointer
Memori	Lebih efisien	Membutuhkan tambahan pointer

Kelebihan Linked List

- Ukuran data dapat bertambah secara dinamis.
- Insert dan delete lebih fleksibel.
- Tidak membutuhkan memori yang berurutan.
- Cocok untuk data yang sering berubah.

Kekurangan Linked List

- Tidak dapat mengakses data secara langsung menggunakan index.
- Membutuhkan memori tambahan untuk pointer.
- Proses pencarian lebih lambat.
- Struktur program lebih kompleks dibandingkan Array.

Operasi Insert pada Linked List

Pada operasi insert, Node baru dibuat kemudian dihubungkan dengan Node lain.

Contoh:

Sebelum:

$A \rightarrow B \rightarrow C$

Insert X:

$A \rightarrow B \rightarrow X \rightarrow C$

Pointer next dari Node sebelumnya diarahkan ke Node baru.

Operasi Delete pada Linked List

Pada operasi delete, Node yang ingin dihapus dilewati dengan mengubah pointer.

Contoh:

Sebelum:

$A \rightarrow B \rightarrow C$

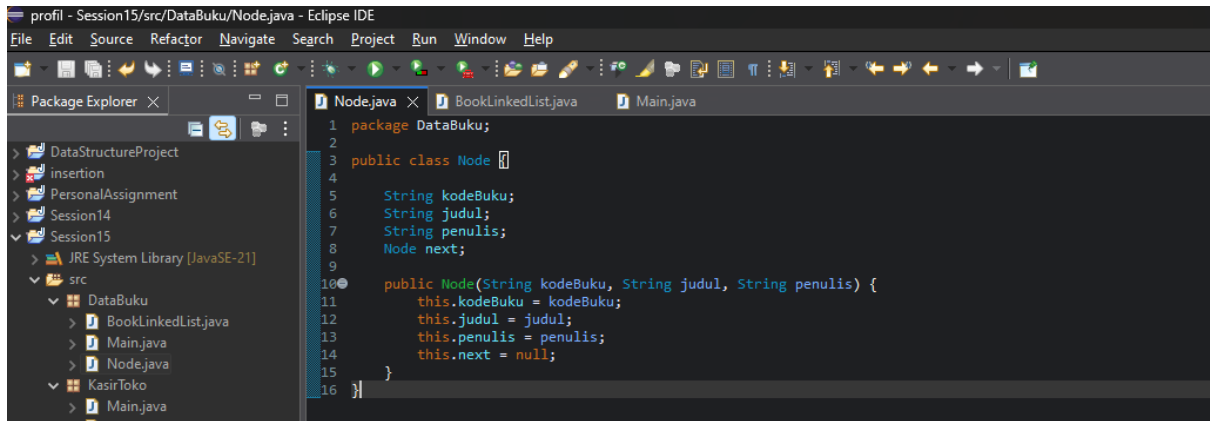
Hapus B:

$A \rightarrow C$

Pointer next dari A langsung diarahkan ke C.

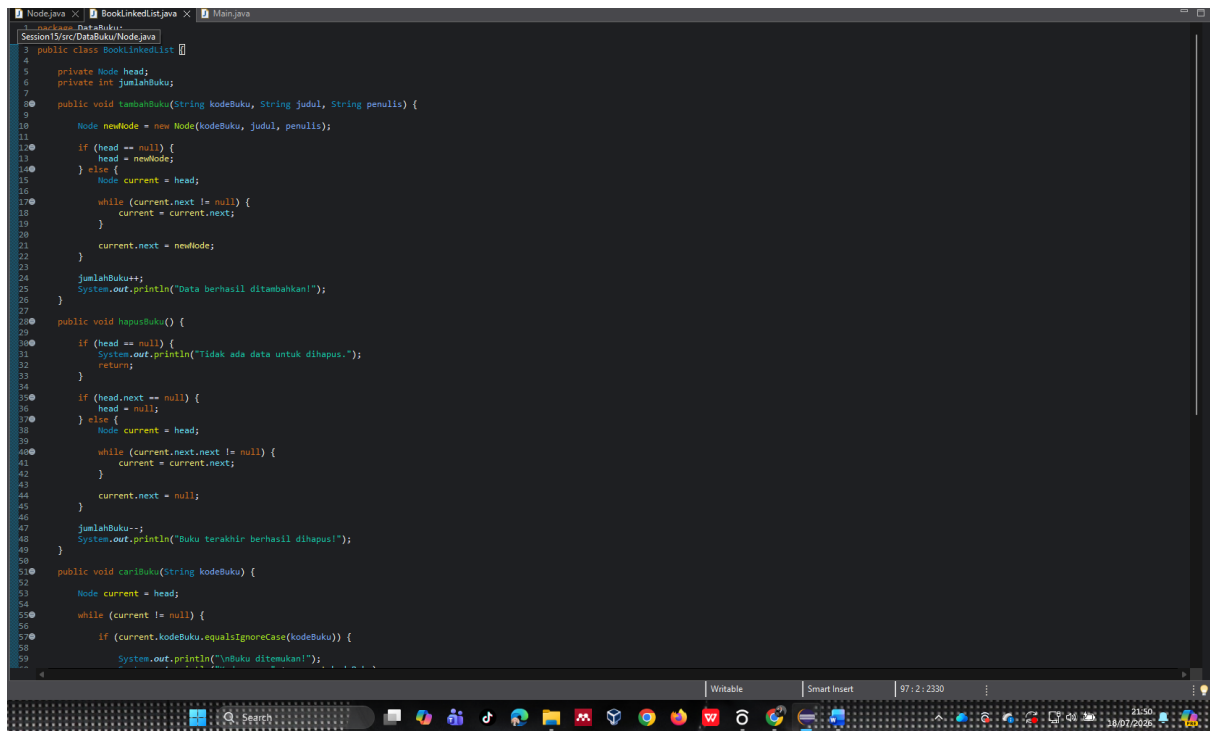
Pertanyaan Ke- 2

Node.java

A screenshot of the Eclipse IDE interface. The Package Explorer on the left shows a project named 'DataStructureProject' with a package 'src' containing 'DataBuku'. Under 'DataBuku', there are three files: 'BookLinkedList.java', 'Main.java', and 'Node.java'. The 'Node.java' file is selected and its code is displayed in the main editor. The code defines a 'Node' class within the 'DataBuku' package. It has three attributes: 'kodeBuku', 'judul', and 'penulis', all of type 'String', and a 'next' attribute of type 'Node'. A constructor 'Node(String kodeBuku, String judul, String penulis)' initializes these attributes and sets 'next' to null.

```
1 package DataBuku;
2
3 public class Node {
4
5     String kodeBuku;
6     String judul;
7     String penulis;
8     Node next;
9
10    public Node(String kodeBuku, String judul, String penulis) {
11        this.kodeBuku = kodeBuku;
12        this.judul = judul;
13        this.penulis = penulis;
14        this.next = null;
15    }
16 }
```

BooklinkedList.java

A screenshot of the Eclipse IDE interface showing the 'BookLinkedList.java' file. The code defines a 'BookLinkedList' class with three methods: 'tambahBuku', 'hapusBuku', and 'cariBuku'. The 'tambahBuku' method adds a new node to the end of the list. The 'hapusBuku' method removes a node from the list. The 'cariBuku' method searches for a node with a specific 'kodeBuku'. The code uses a 'head' pointer to track the first node and a 'current' pointer to traverse the list.

```
1 package DataBuku;
2
3 public class BookLinkedList {
4
5     private Node head;
6     private int jumlahBuku;
7
8     public void tambahBuku(String kodeBuku, String judul, String penulis) {
9
10        Node newNode = new Node(kodeBuku, judul, penulis);
11
12        if (head == null) {
13            head = newNode;
14        } else {
15            Node current = head;
16
17            while (current.next != null) {
18                current = current.next;
19            }
20
21            current.next = newNode;
22        }
23
24        jumlahBuku++;
25        System.out.println("Data berhasil ditambahkan!");
26    }
27
28    public void hapusBuku() {
29
30        if (head == null) {
31            System.out.println("Tidak ada data untuk dihapus.");
32            return;
33        }
34
35        if (head.next == null) {
36            head = null;
37        } else {
38            Node current = head;
39
40            while (current.next.next != null) {
41                current = current.next;
42            }
43
44            current.next = null;
45        }
46
47        jumlahBuku--;
48        System.out.println("Buku terakhir berhasil dihapus!");
49    }
50
51    public void cariBuku(String kodeBuku) {
52
53        Node current = head;
54
55        while (current != null) {
56
57            if (current.kodeBuku.equalsIgnoreCase(kodeBuku)) {
58                System.out.println("\nBuku ditemukan!");
59            }
60        }
61    }
62 }
```

```

50
51
52 public void cariBuku(String kodeBuku) {
53
54     Node current = head;
55
56     while (current != null) {
57
58         if (current.kodeBuku.equalsIgnoreCase(kodeBuku)) {
59
60             System.out.println("\nBuku ditemukan!");
61             System.out.println("Kode : " + current.kodeBuku);
62             System.out.println("Judul : " + current.judul);
63             System.out.println("Penulis : " + current.penulis);
64
65             return;
66
67         }
68         current = current.next;
69     }
70     System.out.println("Buku tidak ditemukan.");
71 }
72
73 public void tampilkanSemuaBuku() {
74
75     if (head == null) {
76         System.out.println("Belum ada data buku.");
77         return;
78     }
79
80     Node current = head;
81
82     System.out.println("\nDaftar Buku:");
83
84     while (current != null) {
85
86         System.out.println(
87             "Kode : " + current.kodeBuku
88             + " | Judul: " + current.judul
89             + " | Penulis: " + current.penulis
90         );
91
92         current = current.next;
93     }
94     System.out.println("\nJumlah Buku: " + jumlahBuku);
95 }
96
97

```

Output

```

===== DATA SISTEM BUKU =====
1. Tambah Buku
2. Hapus Buku
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 1
Masukkan Kode Buku: BK101
Masukkan Judul: Struktur Data
Masukkan Penulis: Michael Goodrich
Data berhasil ditambahkan!

===== DATA SISTEM BUKU =====
1. Tambah Buku
2. Hapus Buku
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu: 4

Daftar Buku:
Kode : BK101 | Judul: Struktur Data | Penulis: Michael Goodrich

Jumlah Buku: 1

===== DATA SISTEM BUKU =====
1. Tambah Buku
2. Hapus Buku
3. Cari Buku
4. Lihat Semua Buku
5. Keluar
Pilih menu:

```

Pertanyaan Ke- 3

- Perbedaan Stack dan Queue

Stack

Stack menggunakan prinsip:

LIFO (Last In First Out)

Artinya, data yang terakhir masuk akan keluar terlebih dahulu.

Contoh:

Push A

Push B

Push C

Pop → C

Operasi utama:

- push() → menambahkan data
- pop() → menghapus data paling atas
- peek() → melihat data paling atas

Contoh nyata Stack

- Tombol Undo
- Riwayat browser
- Call Stack pada program
- Fitur Back pada aplikasi

Queue

Queue menggunakan prinsip:

FIFO (First In First Out)

Artinya, data yang pertama masuk akan keluar terlebih dahulu.

Contoh:

A → B → C

Dequeue → A

Operasi utama:

- enqueue() → menambahkan data ke belakang
- dequeue() → menghapus data dari depan
- peek() → melihat data paling depan

Contoh nyata Queue

- Antrian kasir
- Antrian printer
- Antrian pelanggan bank
- Sistem pemrosesan tugas

Pertanyaan Ke-4

Node.java

```
Node.java X
1 package KasirToko;
2 public class Node {
3
4     String nomorAntrian;
5     String namaPelanggan;
6     double totalBelanja;
7
8     Node next;
9
10    public Node(
11        String nomorAntrian,
12        String namaPelanggan,
13        double totalBelanja
14    ) {
15        this.nomorAntrian = nomorAntrian;
16        this.namaPelanggan = namaPelanggan;
17        this.totalBelanja = totalBelanja;
18        this.next = null;
19    }
20 }
```

Queue.java

```
Session15/src/KasirToko/Node.java
2
3 public class Queue {
4
5     private Node front;
6     private Node rear;
7     private int jumlahPelanggan;
8
9     // Menambahkan pelanggan ke antrian
10    public void enqueue(
11        String nomorAntrian,
12        String namaPelanggan,
13        double totalBelanja
14    ) {
15
16        Node newNode = new Node(
17            nomorAntrian,
18            namaPelanggan,
19            totalBelanja
20        );
21
22        if (rear == null) {
23            front = rear = newNode;
24        } else {
25            rear.next = newNode;
26            rear = newNode;
27        }
28
29        jumlahPelanggan++;
30
31        System.out.println(
32            "Data pelanggan ditambahkan ke antrian!"
33        );
34    }
35
36    // Menghapus pelanggan dari depan
37    public Node dequeue() {
38
39        if (front == null) {
40            return null;
41        }
42
43        Node temp = front;
44        front = front.next;
45
46        if (front == null) {
47            rear = null;
48        }
49
50        temp.next = null;
51        jumlahPelanggan--;
52
53        return temp;
54    }
55 }
```

```

56 // Menampilkan antrian
57 public void tampilkanAntrian() {
58     if (front == null) {
59         System.out.println("Antrian kosong.");
60         return;
61     }
62     Node current = front;
63     System.out.println("\n=== ANTRIAN PELANGGAN ===");
64     while (current != null) {
65         System.out.println(
66             current.nomorAntrian
67             + " | "
68             + current.namaPelanggan
69             + " | Rp "
70             + current.totalBelanja
71             );
72         current = current.next;
73     }
74 }
75
76 public int getJumlahPelanggan() {
77     return jumlahPelanggan;
78 }
79 }

```

Stack.java

```

1 package KasirToko;
2 Session19/src/KasirToko/Node.java
3 public class Stack {
4
5     private Node top;
6
7     // Menambahkan transaksi ke Stack
8     public void push(Node transaksi) {
9
10         transaksi.next = top;
11         top = transaksi;
12     }
13
14     // Menampilkan riwayat transaksi terbaca ke lama
15     public void display() {
16
17         if (top == null) {
18             System.out.println("Belum ada riwayat transaksi.");
19             return;
20         }
21
22         Node current = top;
23
24         System.out.println("\n=== RIWAYAT TRANSAKSI ===");
25
26         while (current != null) {
27
28             System.out.println(
29                 current.nomorAntrian
30                 + " | "
31                 + current.namaPelanggan
32                 + " | Rp "
33                 + current.totalBelanja
34             );
35
36             current = current.next;
37         }
38     }
39 }

```

Output

```

Main [D:\Java Application] C:\Users\Aditya\p2\pooh\plugins\org.eclipse.jdt.launching.hotspot.jre.full.win32.x86_64-21.0.10.v20230205-0638\jre\bin\javaw.exe (Jul-18, 2026, 10:00:51 PM elapsed:0:01:41) [pid: 14012]
WARNING:
Masukkan Nomor Antrian: AB01
Masukkan Nama Pelanggan: Aditya
Masukkan Total Belanja: 50000
Data pelanggan ditambahkan ke antrian!

=== SISTEM KASIR TOKO ===
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 4
Belum ada riwayat transaksi.

=== SISTEM KASIR TOKO ===
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 2
Melayani pelanggan AB01 (Aditya)
Transaksi diisipkan ke riwayat.

=== SISTEM KASIR TOKO ===
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu: 4

=== RIWAYAT TRANSAKSI ===
AB01 | Aditya | Rp 50000.0

=== SISTEM KASIR TOKO ===
1. Tambah Antrian
2. Layani Pelanggan
3. Tampilkan Antrian
4. Lihat Riwayat Transaksi
5. Keluar
Pilih menu:

```

Berikut Link Github: <https://github.com/Sechan24/Session-15.git>